

《Java 并发、分布式与微服务》期末复习资料

一、初识微服务

- **核心概念**：微服务架构是一种将单一应用程序划分为一组小型、独立服务的方法，每个服务围绕特定的业务能力构建。
- **架构挑战与缺点**
 - a) 服务间远程调用会带来网络延迟和复杂性。
 - b) 分布式系统面临数据一致性（分布式事务）的挑战。
 - c) 系统整体运维、部署和监控的复杂度显著增加。
- **设计原则与反模式**：
 - **不推荐**所有服务使用完全相同的技术栈（微服务倡导技术异构）。
 - **不推荐**微服务之间共享数据库（应遵循数据库私有化原则，通过 API 或消息通信）。
- **消息中间件的核心目的**：实现服务间的异步解耦、流量削峰填谷、处理分布式事务的最终一致性。（**注意**：引入消息中间件**不是**为了解决服务调用的链路追踪问题）。

二、Spring Cloud Alibaba 之注册中心 (Nacos)

- **双重定位**：Nacos 既可以作为**服务注册与发现中心**，也可以作为**分布式配置中心**使用。
- **启动模式**：Nacos 服务端启动时，**默认是以单机模式 (standalone) 启动的**，而非集群模式。
- **实例类型**：支持**临时实例** (Ephemeral，默认，心跳机制) 和**持久化实例** (Persistent)。支持健康检查机制。

- **配置管理隔离性：**
 - **Namespace (命名空间)：**用于环境隔离（如 dev, test, prod）。不同的 Namespace 下，可以有相同的 Data ID。
 - **Data ID：**在**同一个** Namespace 下，Data ID 的命名必须是唯一的。
- **动态刷新：**通常配合 @RefreshScope 注解，实现配置中心属性变更时，Bean 中配置值的动态刷新。

三、Spring Cloud Alibaba 之负载均衡 (LoadBalancer)

- **负载均衡类型：**Spring Cloud LoadBalancer 属于**客户端负载均衡**。它由服务消费者维护服务提供者列表并在本地进行路由选择，**不需要**在请求入口前部署独立的服务端负载均衡器。
- **默认策略：**默认实现了**轮询策略** (RoundRobinRule)，将请求按顺序依次分配给服务器。
- **服务发现联动：**与 Nacos 结合使用时，能自动维护服务实例列表。当某个服务实例下线后，消费者能及时感知并停止向其发送请求，避免调用失败。

四、Spring Cloud Alibaba 之服务通信 (RestTemplate、OpenFeign、RocketMQ)

1. RestTemplate

- Spring 框架提供的**同步** HTTP 客户端。
- 可以通过与服务发现组件（如 LoadBalancer）结合，直接使用**服务名**进行调用，而不仅限于 IP 和端口。
- 支持 GET、POST 等多种请求类型，使用 getObject() 等方法时，返回值会自动反序列化为指定类型。

2. OpenFeign

- **定位**：声明式的 HTTP 客户端，简化了远程服务调用的代码编写（无需手动编写底层 HTTP 请求代码）。
- **核心注解**：
 - `@EnableFeignClients`：标注在启动类上，用于开启 Feign 客户端功能，默认会扫描主程序类所在包及其子包下所有带有 `@FeignClient` 的接口，并生成动态代理。
 - `@FeignClient`：标注在接口上，将其标识为 Feign 客户端。`name` 属性用于指定远程服务的名称；可通过 `fallback` 或 `fallbackFactory` 属性指定服务降级后的处理类。
 - `@PostMapping` / `@GetMapping` 等：用于映射具体的 HTTP 请求路径。

3. RocketMQ 与 Spring Cloud Stream

- **RocketMQ**：由阿里巴巴开源并捐赠给 Apache 基金会的高性能消息中间件。
- **Stream 编程模型**：基于三个核心函数式接口构建：
 - `Supplier`：用于**发送/生产**消息。
 - `Consumer`：用于**接收/消费**消息。
 - `Function`：用于接收消息、处理并**转发**消息。
- **消息分组 (Consumer Group)**：同一 Group 内的多个消费者实例会以**竞争关系**消费消息，每条消息只会被其中一个实例消费，从而保证了消息消费的**唯一性**和负载均衡。

五、Spring Cloud Alibaba 之流量控制 (Sentinel)

- **核心规则与作用**：

- **流量控制**：可基于 QPS 或并发线程数进行限制。
- **熔断降级**：可基于**平均响应时间（RT）、异常比例或异常数**等指标触发，防止雪崩。
- **系统保护**：针对系统整体维度（如 Load、CPU 使用率）的保护，而非单个 API 粒度。
- **热点参数**：针对经常访问的热点参数进行限流，而非对所有参数统一限流。
- **核心注解**：@SentinelResource，用于定义资源。其 blockHandler 属性可指定处理限流或降级逻辑的方法。
- **架构认知**：Sentinel 控制台（Dashboard）主要用于规则配置和监控，**不是**规则判断的核心执行引擎（执行引擎在应用端本地），且控制台不必强制随应用一起部署。

六、Spring Cloud Alibaba 之服务网关 (Gateway)

- **路由 (Route) 组成**：由 **ID**（唯一标识）、**断言 (Predicate)**（匹配条件）和 **过滤器 (Filter)**（处理逻辑）三部分组成。
- **内置断言工厂 (Route Predicate Factories)**（至少掌握 4 种）：
 - d) PathRoutePredicateFactory：基于请求路径进行匹配。
 - e) MethodRoutePredicateFactory：基于 HTTP 请求方法（GET/POST 等）匹配。
 - f) BeforeRoutePredicateFactory：匹配在指定时间**之前**发起的请求。
 - g) AfterRoutePredicateFactory：匹配在指定时间**之后**发起的请求。
 - h) BetweenRoutePredicateFactory：匹配在两个指定时间**之间**发起的请求。
 - i) HeaderRoutePredicateFactory：基于请求头进行匹配。
- **过滤器 (Filter)**：
 - **局部过滤器 (GatewayFilter)**：针对特定路由生效。如 AddRequestHeader

(添加请求头)、AddRequestParameter (添加请求参数)、PrefixPath (在转发时**添加**指定前缀, 注意不是移除)。

- **全局过滤器 (GlobalFilter)** : 对所有路由自动生效 (如内置的 LoadBalancerClientFilter) 。自定义全局过滤器必须同时实现 GlobalFilter 和 Ordered 接口。

七、Spring Cloud Alibaba 之分布式事务管理 (Seata)

- **核心角色:**

- **TC (Transaction Coordinator)**: 事务协调者, 负责协调和管理全局事务。
- **TM (Transaction Manager)**: 事务管理器, 负责开启、提交或回滚全局事务。
- **RM (Resource Manager)**: 资源管理器, 即具体执行本地事务的分支参与者。

- **四种事务模式:**

- j) **AT 模式 (默认)** : 基于两阶段提交 (2PC) 优化。一阶段执行业务 SQL 并提交, 同时生成数据快照 (undo log) , **释放本地锁但保留全局锁**; 二阶段由 TC 协调, 若失败则利用快照自动补偿。对业务代码侵入小。the2. **TCC 模式**: Try-Confirm-Cancel。对业务代码**有侵入**, 需要开发者手动实现这三个接口。
 - k) **Saga 模式**: 适用于**长事务**, 通过状态机或补偿服务实现, **无代码侵入**。
 - l) **XA 模式**: 强一致性, 由数据库层面保证, 但可能存在**性能瓶颈**。
- **存储模式**: file (单机/测试)、db (数据库, 适合生产集群)、redis (性能好, 但因内存特性**存在数据丢失风险**) 。

八、Spring Cloud Alibaba 之分布式链路追踪 (SkyWalking)

- **定位**: 一款开源的分布式应用性能监控 (APM) 和链路追踪系统。
- **多语言支持**: **不仅支持 Java**, 还支持多种其他编程语言 (如 Go, Python, Node.js

等)。

- **核心概念：**

- **Trace**：表示一个完整的请求链路。

- **Span**：表示链路中的一个工作单元或节点（如一次 RPC 调用或数据库查询）。

- **核心组件：Agent (探针)**。部署在应用程序内部，用于无侵入地收集调用链数据并上报给服务端。

- **集成与存储**：支持与 Spring Cloud Gateway 等组件无缝集成；数据存储支持多种方式（如 Elasticsearch 等），**可以持久化**，并非只能使用内存。

九、部分代码

1. Feign 接口定义：

```
@FeignClient(name = "product-service") //远程服务名

public interface ProductClient {

    @PostMapping("/api/products")    //接口路径

    List<Product> getProducts(@RequestBody List<Long> ids); // 注意

    @RequestBody

}
```

2. 启动类与配置文件：

```
@SpringBootApplication

@EnableFeignClients           //开启 Feign

public class OrderApplication { ... }

spring:
```

```
application:

  name: order-service          # 当前应用名

cloud:

  nacos:

    discovery:

      server-addr: localhost:8848    # Nacos 默认端口 8848
```

3. Service 层:

```
@Service

public class OrderService {

    @Autowired

    private ProductClient productClient;

    @Autowired

    private OrderMapper orderMapper;    //注入 Mapper

    public OrderWithProduct getOrderDetails(Long orderId) {

        // ... 获取 order 和 ids ...

        List<Product> products = productClient.getProducts(productId); // 调用

        Feign

        return new OrderWithProduct(order, products);    //封装返回

    }

}
```

4. Controller 层:

```
@RestController

@RequestMapping("/api/orders")

public class OrderController {

    @GetMapping("/{orderId}")

    public OrderWithProduct getOrder(@PathVariable Long orderId)

    { // @PathVariable

        return orderService.getOrderDetails(orderId);        //调用 Service

    }

}
```